National Textile University

Department of Computer Science

Subject:

Mobile application and development

Submitted To:

Sir Rehan Sarwar

Submitted By:

Noor ul Ain

Registration No:

23-NTU-CS-1221

Lab no:

8

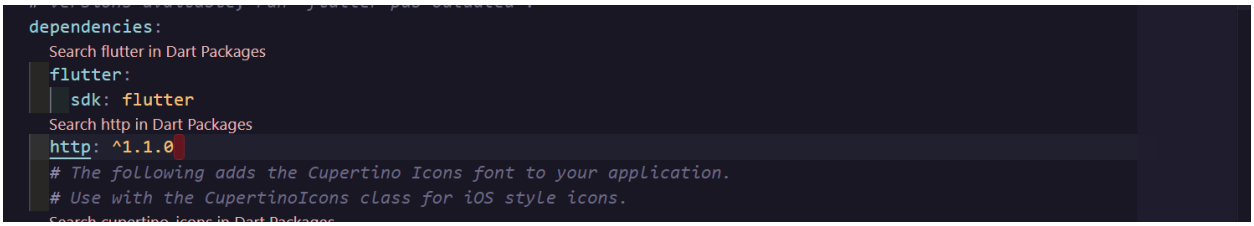
Semester:

6th

Flutter Weather Fetcher App

Part 1: Setup and API Key

- Generate an API Key
- Initialize the Flutter Project
- Add Dependencies



Part 2: The Data Model

```dart
class Weather {
  final String cityName;
  final double temperature;
  final String description;
  Weather({
    required this.cityName,
    required this.temperature,
    required this.description,
  });
   // Factory constructor to parse JSON
  factory Weather.fromJson(Map<String, dynamic> json) {
    return Weather(
      cityName: json['name'],
      temperature: json['main']['temp'].toDouble(),
      description: json['weather'][0]['description'],
    );
  }
}
```

Part 3: The API Service

```dart
import 'dart:convert';
import 'package:http/http.dart' as http;
import 'weather_model.dart';
class WeatherService {
  // Replace with your actual API key
  static const String apiKey = '67ed64464cc45c1f6e5325f42bf7d24b';
  // Using London's coordinates as default for the lab
  static const double lat = 51.5074;
  static const double lon = -0.1278;
  Future<Weather> fetchWeather() async {
    // We add &units=metric to get Celsius instead of Kelvin
    final url = Uri.parse(
    'https://api.openweathermap.org/data/2.5/weather?lat=$lat&lon=$lon&appid=$apiKey&units=metric'
);
    final response = await http.get(url);
    if (response.statusCode == 200) {
      // Parse the JSON data
      final jsonResponse = jsonDecode(response.body);
      return Weather.fromJson(jsonResponse);
    } else {
      throw Exception('Failed to load weather data');
    }
  }
}
```

Part 4: Building the User Interface

```dart
import 'package:flutter/material.dart';
import 'weather_model.dart';
import 'weather_Service.dart';
void main() {
  runApp(const WeatherApp());
}
class WeatherApp extends StatelessWidget {
  const WeatherApp({super.key});
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'Weather Lab',
      theme: ThemeData(primarySwatch: Colors.blue),
       home: const WeatherScreen(),
    );}}
class WeatherScreen extends StatefulWidget {
  const WeatherScreen({super.key});
  @override
  State<WeatherScreen> createState() => _WeatherScreenState();
}
class _WeatherScreenState extends State<WeatherScreen> {
  final WeatherService _weatherService = WeatherService();

  late Future<Weather> _weatherFuture;
  @override
  void initState() {
    super.initState();
    // Trigger the API call when the screen loads
    _weatherFuture = _weatherService.fetchWeather(); }

  void _refreshWeather() {
    setState(() {
      _weatherFuture = _weatherService.fetchWeather();
    }); }
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Current Weather'),
        actions: [
          IconButton(
            icon: const Icon(Icons.refresh),
            onPressed: _refreshWeather,
          )
        ], ),
       body: Center(
        child: FutureBuilder<Weather>(
          future: _weatherFuture,
          builder: (context, snapshot) {
            // State 1: Still loading
            if (snapshot.connectionState == ConnectionState.waiting) {
              return const CircularProgressIndicator(); }
            // State 2: Error occurred
            else if (snapshot.hasError) {
              return Text('Error: ${snapshot.error}');}
            // State 3: Data successfully fetched
            else if (snapshot.hasData) {
              final weather = snapshot.data!;
              return Column(
                mainAxisAlignment: MainAxisAlignment.center,
                children: [
                  Text(
                    weather.cityName,
                    style: const TextStyle(fontSize: 40, fontWeight:
FontWeight.bold),),
                  const SizedBox(height: 10),
                  Text(
                    '${weather.temperature.toStringAsFixed(1)}°C',
                    style: const TextStyle(fontSize: 60),
                  ),
                  const SizedBox(height: 10),
                  Text(
                    weather.description.toUpperCase(),
                    style: const TextStyle(fontSize: 24, color: Colors.grey),
                  ),
                ],
              ); }
            // Fallback state
            return const Text('No data available'); },), ),); }
}
```

Part 5: Lab Execution & Bonus Challenges

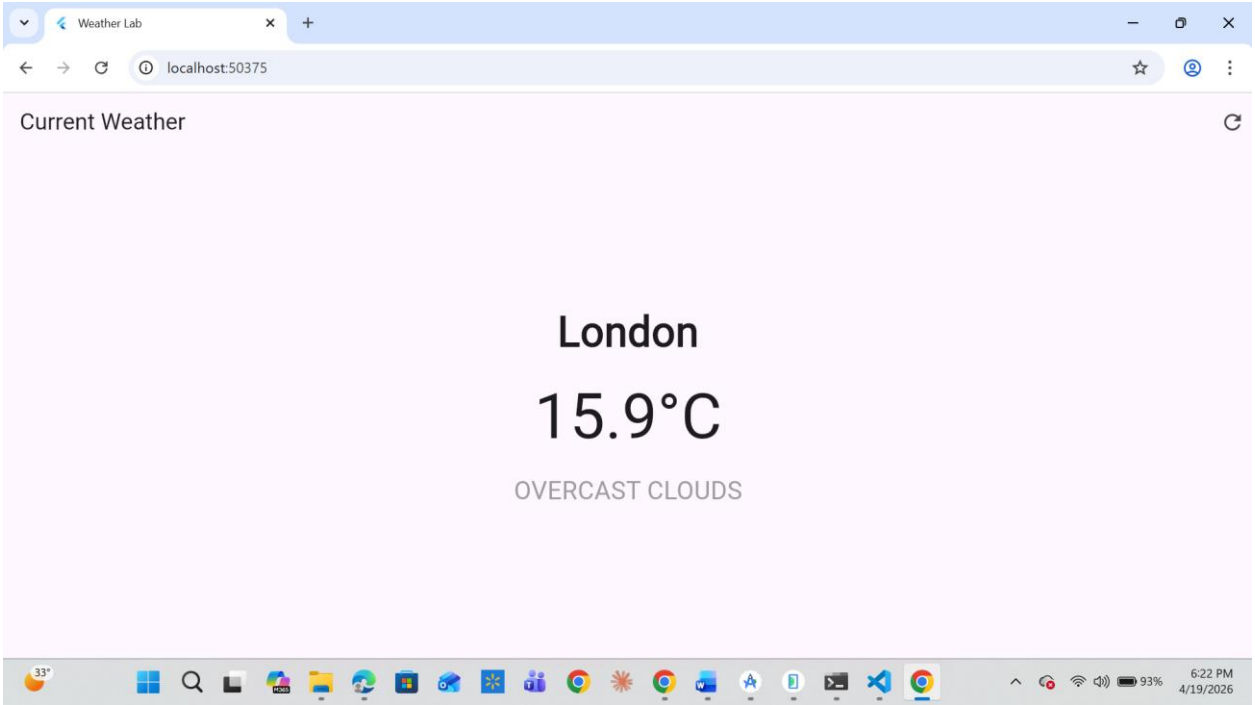
Execution:

1. Insert your actual API key into weather_service.dart

```
weather_Service.dart > WeatherService
import 'dart:convert';    The file name 'weather_Service.dart' isn't a lower_case_with_under
import 'package:http/http.dart' as http;
import 'weather_model.dart';
class WeatherService {
  // Replace with your actual API key
  static const String apiKey = '67ed64464cc45c1f6e5325f42bf7d24b';
  // Using London's coordinates as default for the lab
  static const double lat = 51.5074;
  static const double lon = -0.1278;
  Future<Weather> fetchWeather() async {
    // We add &units=metric to get Celsius instead of Kelvin
    final url = Uri.parse(
```

2. Run the app (Flutter run).
3. You should see a loading spinner briefly, followed by the current temperature, city name, and condition for London.

Output:



BONUS CHALLENGES

- **Challenge 1 (Dynamic Inputs):** Instead of hardcoding `lat` and `lon` in the `fetchWeather` method, add a Flutter `TextField` so the user can input the latitude and longitude dynamically and press a button to search.

Code:

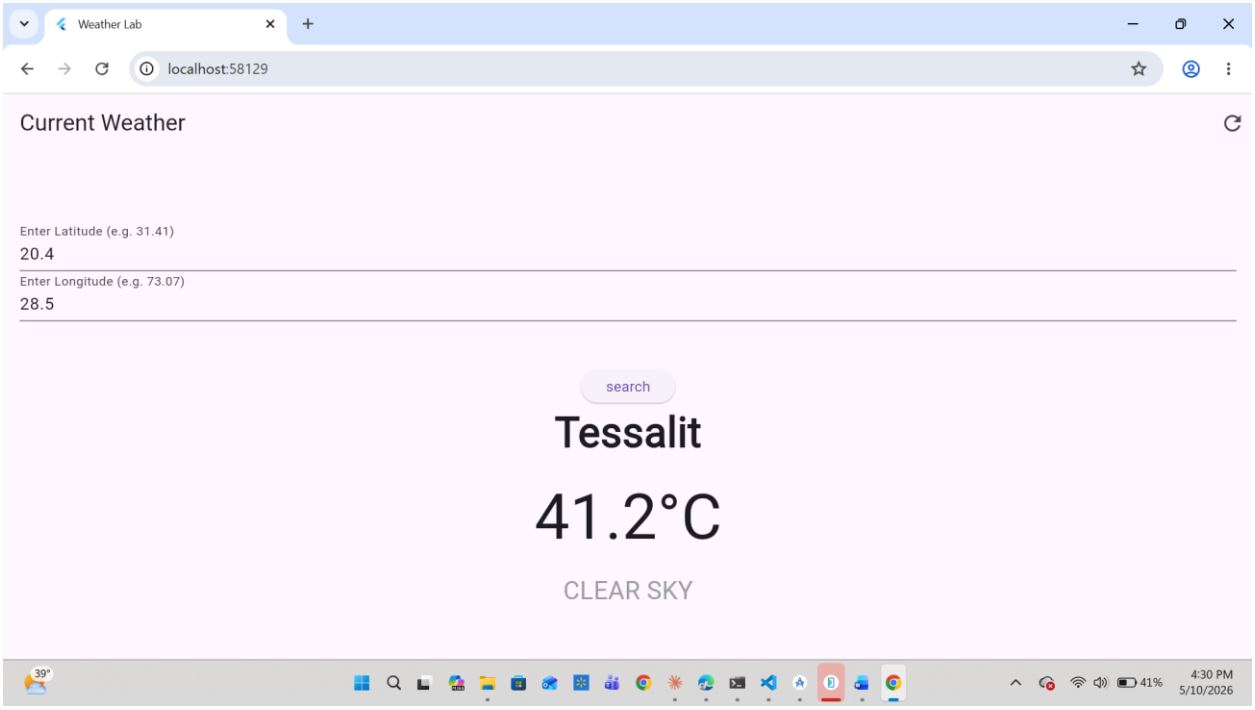
Main.dart file:

```dart
@override
void initState() {
  super.initState();

  _weatherFuture = _weatherService.fetchWeather(51.5074, -0.1278);
}
  void _refreshWeather() {
  // 1. Get the text from controllers
  // 2. Convert to double (if empty or invalid, use London as fallback)
  double lat = double.tryParse(_latController.text) ?? 51.5074;
  double lon = double.tryParse(_lonController.text) ?? -0.1278;

  setState(() {
    // 3. This triggers the FutureBuilder to run again with NEW values
    _weatherFuture = _weatherService.fetchWeather(lat, lon);
  });
}
```

Dynamic text fields:

```dart
padding: const EdgeInsets.all(16.0),
 child: Column(
   children: [
     TextField(
       controller: _latController,
       keyboardType: const TextInputType.numberWithOptions(decimal: true),
       decoration: const InputDecoration(labelText: "Enter Latitude (e.g. 31.41)"),
     ),
     TextField(
       controller: _lonController,
       keyboardType: const TextInputType.numberWithOptions(decimal: true),
       decoration: const InputDecoration(labelText: "Enter Longitude (e.g. 73.07)"),
     ),
     const SizedBox(height: 10),
     /* ElevatedButton(
       onPressed: _refreshWeather, // This will now trigger the update
       child: const Text("Search"),
     ), */
```

OUTPUT:



- **Challenge 2 (Geocoding API):** The OpenWeather API documentation recommends using their Geocoding API to turn a City Name into Lat/Lon. Create a two-step API call where the user types "New York", the app fetches the Lat/Lon using the Geocoder endpoint, and then fetches the weather.

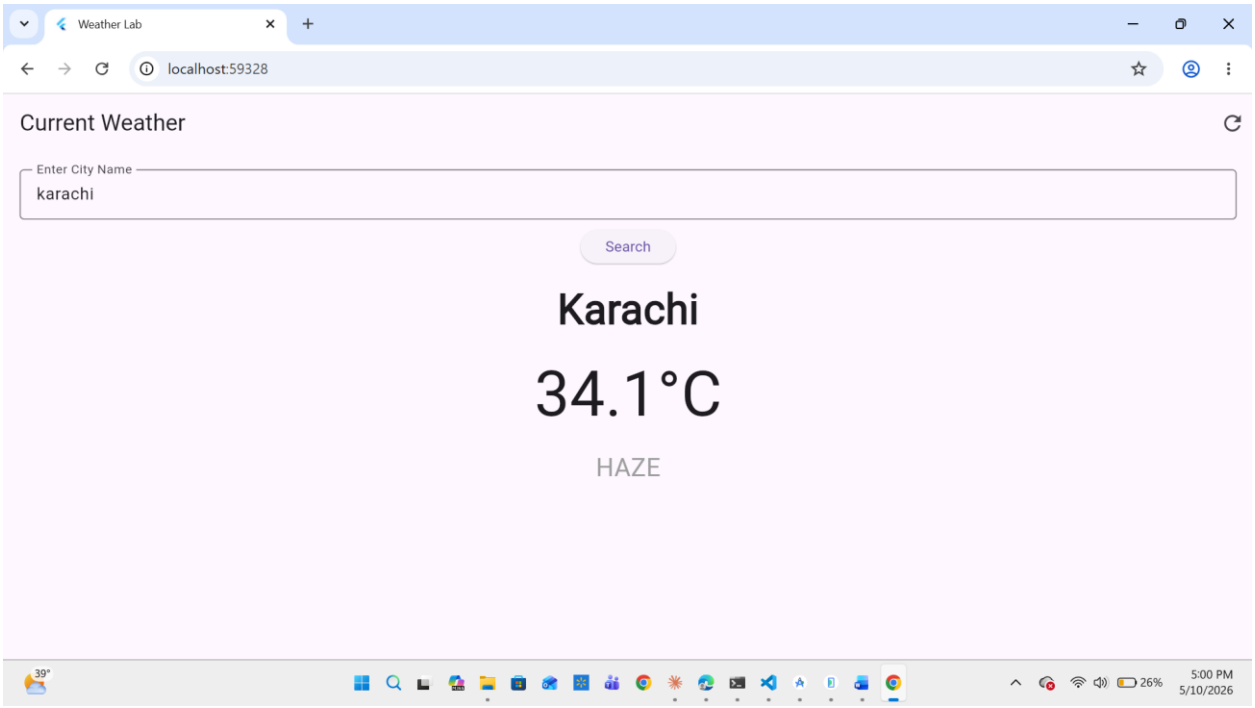
Adding function to weather_service. Dart class file:

```dart
Future<Map<String, double>> getCoordinates(String cityName) async {
  final geoUrl = Uri.parse(
      'https://api.openweathermap.org/geo/1.0/direct?q=$cityName&limit=1&appid=$apiKey');

  final response = await http.get(geoUrl);

  if (response.statusCode == 200) {
    final List data = jsonDecode(response.body);
    if (data.isNotEmpty) {
      return {
        'lat': data[0]['lat'].toDouble(),
        'lon': data[0]['lon'].toDouble(),
      };
    } else {
      throw Exception('City not found');
    }
  } else {
    throw Exception('Failed to fetch coordinates');
  }
}
}
```

Changes in Main.dart:

```dart
@override
  void dispose() {
    _cityController.dispose();
    super.dispose();
  }

  // The logic for Challenge 2: Two-step API call
  Future<Weather> _performTwoStepSearch(String city) async {
    // 1. Get Coordinates first from the Geocoding API
    final coords = await _weatherService.getCoordinates(city);

    // 2. Then get Weather using those coordinates
    return await _weatherService.fetchWeather(coords['lat']!, coords['lon']!);
  }

  void _searchByCity() {
    String city = _cityController.text.trim();
    if (city.isNotEmpty) {
      setState(() {
        _weatherFuture = _performTwoStepSearch(city);
      });
    }
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Current Weather'),
        actions: [
          IconButton(
            icon: const Icon(Icons.refresh),
            onPressed: _searchByCity,
          )
        ],
      ),
```

OUTPUT:


Weather Lab
localhost:59328
Current Weather
Enter City Name
karachi
Search
Karachi
34.1°C
HAZE
39°
26%
5:00 PM
5/10/2026

- **Challenge 3 (UI Enhancement):** Use the `icon` string returned in the JSON (`json['weather'][0]['icon']`) to display the official OpenWeatherMap icon in the UI using `Image.network('https://openweathermap.org/img/wn/{icon_code}@2x.png')`.

Updated the model.dart file:

```dart
class Weather {
  final String cityName;
  final double temperature;
  final String description;
  final String iconCode;
  Weather({
    required this.cityName,
    required this.temperature,
    required this.description,
    required this.iconCode,
  });
  // Factory constructor to parse JSON
  factory Weather.fromJson(Map<String, dynamic> json) {
    return Weather(
      cityName: json['na
      temperature: json['main']['temp'].toDouble(),
      description: json['weather'][0]['description'],
      iconCode: json['weather'][0]['icon'],
    );
  }
}
```

Type: String

Added image network widget in column widget in main.dart file:

```
Image.network(
    'https://openweathermap.org/img/wn/${weather.iconCode}@2x.png',
    width: 100,
    height: 100,
    errorBuilder: (context, error, stackTrace) => const Icon(Icons.wb_sunny, size: 50),
  ),

  Text(
    '${weather.temperature.toStringAsFixed(1)}°C',
    style: const TextStyle(fontSize: 60),
  ),
```

OUTPUT: